

The Definitive Guide to Apache Spark

From Beginner to Pro: Architecture, Dockerization, Kubernetes
Orchestration, and Real-World Implementation

Data Engineering Masterclass

October 6, 2026

Contents

1 Part 1: The Absolute Beginner's Guide to Apache Spark

1.1 What is Apache Spark?

Apache Spark is a lightning-fast, open-source unified analytics engine for large-scale data processing and machine learning. Originally developed at UC Berkeley's AMPLab in 2009, Spark was designed to address the limitations of Hadoop MapReduce—specifically, its heavy reliance on disk I/O, which made iterative algorithms and interactive data mining extremely slow.

Spark achieves its speed through **in-memory computing**. By caching data in RAM rather than writing it back to disk after every step, Spark can be up to 100x faster than Hadoop for certain workloads.

1.2 The Evolution of Spark Data Structures

- **RDD (Resilient Distributed Dataset):** The fundamental, low-level data structure of Spark. RDDs are immutable, fault-tolerant collections of objects distributed across a cluster. They offer fine-grained control but lack optimization engines.
- **DataFrame:** Introduced in Spark 1.3, DataFrames organize data into named columns, similar to a table in a relational database. They sit on top of the Catalyst Optimizer, which automatically optimizes query plans.
- **Dataset:** Introduced in Spark 1.6, Datasets combine the strong typing and object-oriented programming style of RDDs with the performance optimizations of DataFrames.

1.3 High-Level Spark Architecture

Spark operates on a master-worker architecture. Here are the core components:

Component	Description
Driver Program	The central coordinator. It runs the <code>main()</code> function, creates the <code>SparkContext/SparkSession</code> , and converts user code into execution tasks.
Cluster Manager	Responsible for acquiring resources on the cluster. Spark supports Standalone, Hadoop YARN, Apache Mesos, and Kubernetes.
Executors	Worker nodes that execute the tasks assigned by the Driver. They store data in memory or on disk for caching and return results.

Table 1: Core Spark Components

2 Part 2: Intermediate Spark - Core Execution and Storage

2.1 Transformations vs. Actions

Spark evaluates code **lazily**, meaning it doesn't execute anything until a result is explicitly requested.

- **Transformations:** Operations that return a new DataFrame/RDD (e.g., `select()`, `filter()`, `join()`). They build a logical execution plan (DAG) but do not compute data.
- **Actions:** Operations that return a final result to the driver or write data to storage (e.g., `count()`, `collect()`, `write()`). Calling an action triggers the execution of the entire DAG.

2.2 Wide vs. Narrow Dependencies (The Shuffle)

- **Narrow Dependencies:** Each partition of the parent RDD is used by at most one partition of the child RDD (e.g., `map()`). No data movement across the network is required.
- **Wide Dependencies:** Multiple child partitions may depend on a single parent partition (e.g., `groupByKey()`, `join()`). This requires a **Shuffle**—moving data across the network between executors, which is highly resource-intensive.

3 Part 3: Containerizing Spark with Docker (Real-World Architecture)

Running Spark locally or in a distributed environment requires strict dependency management. Docker encapsulates Spark, Java, Python, and all necessary libraries into isolated containers.

3.1 Real-World Example: The PGS Search Engine Stack

In enterprise environments, Spark rarely operates in isolation. As demonstrated in the **PGS Search Engine** architecture, Spark can be integrated with message brokers, external object storage, and security tools using a unified Docker Compose stack.

In this architecture, we bypass heavy orchestrators like Kubernetes for local/mid-tier deployments and use a **Spark Standalone Cluster** alongside external services:

- **Storage:** LocalStack simulates AWS S3, providing the `crawled-pages` bucket where Spark executors read incoming HTML data.
- **Messaging:** Kafka (in KRaft mode) manages the `scraped_files_topic`, passing events to trigger the ETL process.
- **Security:** A dedicated ClamAV container receives data streams from Spark executors to perform real-time malware scanning.

- **Indexing:** OpenSearch stores the final BM25 text indexes and k-NN vector embeddings generated by the Spark pipeline.

3.2 Docker Compose: Spark Standalone Implementation

Here is how the Spark Master and Workers are defined in a production-grade `docker-compose.yml`:

```

1  spark-master:
2    image: pgs-search-engine/etl:local
3    entrypoint: ["/usr/bin/dumb-init", "--"]
4    command: ["spark-class", "org.apache.spark.deploy.master.Master",
5              "--host", "spark-master", "--port", "7077", "--webui-port", "8080"]
6    ports:
7      - "127.0.0.1:8090:8080" # Spark Master UI
8    restart: unless-stopped
9
10 spark-worker:
11   image: pgs-search-engine/etl:local
12   entrypoint: ["/usr/bin/dumb-init", "--"]
13   command: ["spark-class", "org.apache.spark.deploy.worker.Worker", "
14             spark://spark-master:7077",
15             "--cores", "2", "--memory", "2g", "--work-dir", "/tmp/
16             spark-work"]
17   environment:
18     S3_ENDPOINT: http://s3:4566
19     CLAMD_HOST: clamav
20   deploy:
21     replicas: 2
22   depends_on:
23     spark-master:
24       condition: service_healthy

```

Listing 1: Spark Cluster Docker Compose Definition

4 Part 4: Orchestrating Spark with Apache Airflow & Temporal

While Spark processes the data, you need an orchestrator to manage *when* and *how* those jobs run.

4.1 The Airflow + Temporal Handoff

In the PGS architecture, **Apache Airflow** acts as the high-level schedule manager, while **Temporal** acts as the stateful workflow engine.

1. **Airflow:** Uses a lightweight `LocalExecutor`. DAGs monitor Kafka for incoming sites. Once a batch of 100 sites is reached, Airflow hands the work off to Temporal.
2. **Temporal:** Manages the durable workflow execution.
3. **The Spark Driver:** A long-lived Temporal worker container acts as the Spark Driver. It receives workflows from Temporal, submits jobs to the `spark-master`, and orchestrates embedding generation before writing to OpenSearch.

4.2 The ETL Worker (Driver) Configuration

```
1 etl-worker:
2   image: pgs-search-engine/etl:local
3   command: ["python", "worker.py"]
4   environment:
5     TEMPORAL_ADDRESS: temporal:7233
6     SPARK_MASTER: spark://spark-master:7077
7     SPARK_EXECUTOR_MEMORY: 1g
8     SPARK_EXECUTOR_CORES: "1"
9     EMBEDDING_MODEL_NAME: sentence-transformers/LaBSE
10  volumes:
11    - ./ETL/temporal:/opt/etl/temporal:ro
12    - data-etl-models:/home/airflow/.cache/huggingface
```

Listing 2: ETL Worker Configuration

5 Part 5: Pro Level - Spark on Kubernetes (K8s)

5.1 Spark on Kubernetes Architecture

When you submit a Spark application to Kubernetes natively:

1. The submission contacts the Kubernetes API server.
2. K8s spins up a **Driver Pod**.
3. The Driver Pod requests K8s to spin up **Executor Pods**.
4. Once complete, Executor pods are terminated automatically.

5.2 Running Airflow DAGs on Kubernetes

In a cloud-native setup, you utilize the **Spark Operator** and trigger it via Airflow's `SparkKubernetesOperator`.

```
1 from airflow import DAG
2 from airflow.providers.cncf.kubernetes.operators.spark_kubernetes
   import SparkKubernetesOperator
3
4 with DAG('k8s_spark_operator_dag', schedule_interval=None) as dag:
5     submit_spark_job = SparkKubernetesOperator(
6         task_id='submit_spark_job',
7         namespace='data-processing',
8         application_file='spark-app.yaml',
9         do_xcom_push=True,
10    )
```

Listing 3: Airflow K8s Operator

6 Part 6: Pro Level - Performance Tuning and Best Practices

6.1 Handling Data Skew

- **Salting:** Append a random integer to the skew key to distribute data evenly.
- **Broadcast Joins:** If one table is small, broadcast it to all executors instead of shuffling the large table.

6.2 Adaptive Query Execution (AQE)

Introduced natively in Spark 3.0, AQE re-optimizes the query plan dynamically during runtime.

```
1 spark.conf.set("spark.sql.adaptive.enabled", "true")
2 spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")
3 spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
```

Listing 4: Enabling AQE